

Solution Requirements

Date	16 September 2026
Team ID	SWTID-2026-3976
Project Name	AI FAQ Assistant API
Maximum Marks	4 Marks

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	Users register and log in with name, email & password; passwords are hashed with bcrypt in a Mongoose <code>pre('save')</code> hook, and a JWT is issued via <code>jsonwebtoken</code> and verified against <code>process.env.JWT_SECRET</code> .	High
2	Authorization levels	Route-level protection via <code>middleware/authMiddleware.js</code> : a single <code>protect</code> guard verifies the Bearer token on every private route (create/update/delete FAQ, both AI endpoints); there is no separate admin-only guard or role field in this cycle's implementation.	High
3	External interfaces	Integrates with the Google Gemini API via a dedicated <code>src/services/geminiService.js</code> for two flows: direct question answering (<code>/api/ai/answer</code>) and FAQ-pair generation from a topic (<code>/api/ai/generate-faq</code>).	High
4	Transactions / data processing	Create, read, update, delete and keyword-search FAQ documents (question, answer, category, creator reference); each FAQ is linked to the User who created it via <code>createdBy</code> .	High
5	Reporting	Returns structured JSON for every FAQ, search result and AI-generation call, suitable for any API client (Postman/Thunder Client, or a future frontend) to render.	Medium
6	Business rules	Each user's email must be unique; every FAQ's <code>category</code> must be one of a fixed enum (Technology, Education, Health, Banking, General, defaulting to General); only the authenticated user attached via <code>protect</code> can create, update or delete through the protected FAQ routes.	High
7	Compliance to laws or regulations	Passwords are never stored in plaintext (bcrypt hashing, salt rounds 10) and are excluded from query results via <code>.select('-password')</code> ; the JWT secret and Gemini API key are kept in <code>.env</code> and excluded from version control.	High
8	Other	Centralized error handling (<code>middleware/errorMiddleware.js</code>) returns consistent JSON error responses for Mongoose duplicate-key errors, validation errors, invalid ObjectIds, and any status code set upstream (e.g. a 429 surfaced by the Gemini SDK).	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Auth and FAQ CRUD endpoints should respond quickly against the local/Atlas MongoDB instance; the two Gemini-backed AI endpoints depend on external API latency.	See Performance Testing for measured samples (register, login, profile, AI answer)
2	Scalability	Stateless JWT authentication allows the API to run behind multiple Node.js instances with no shared session state; MongoDB/Mongoose supports horizontal growth of the FAQ collection.	Designed to support growth beyond a single-instance deployment
3	Security & Data Privacy	bcrypt password hashing, JWT-based route protection, and Mongoose schema validation (required fields, enum, unique email) on every write.	No plaintext password storage; all inputs validated before persistence
4	Reliability & Availability	Centralized error-handling middleware plus a dedicated <code>connectDB</code> call in <code>src/server.js</code> before the Express listener starts, with an <code>unhandledRejection</code> handler that closes the server and exits cleanly.	Server logs and exits gracefully rather than hanging on an unhandled promise rejection
5	Usability & Accessibility	Consistent JSON REST responses, verified with a Postman collection (<code>AI_FAQ_Assistant_API.postman_collection.json</code>) covering auth, FAQ and AI-generation endpoints.	Every documented endpoint testable from the shared collection
6	Other (Maintainability)	Modular structure (<code>controllers/ middleware/ models/ routes/ services/ utils/</code>) under <code>src/</code> , so a new endpoint can be added as one controller function, one route, and (if needed) one validator.	New feature addable without touching unrelated modules